AI Tech Stack Recommender

Content-Based Career Recommendation System

Using TF-IDF Vectorization & Cosine Similarity

Documentation v1.0

Table of Contents

1. Project Overview

The AI Tech Stack Recommender is a content-based recommendation engine built with Python. It suggests the most suitable technology careers based on a user's input skills and interests. The system uses Natural Language Processing techniques to match user-provided skills against a curated dataset of 21 tech career paths.

Key Capabilities:

- Interactive CLI-based onboarding for collecting user name and skills
- TF-IDF (Term Frequency - Inverse Document Frequency) vectorization of skill text
- Cosine similarity computation between user input and career skill-sets
- Top-3 career recommendation ranking with similarity scores
- Easily extendable dataset for adding new careers

2. System Architecture

The system follows a simple pipeline architecture:

User Input (Skills) -> TF-IDF Vectorization -> Cosine Similarity Computation -> Ranked Career Recommendations

Components:

1. main.py : Entry point for the CLI application. Handles user onboarding (name, skill input) and displays recommendations.
2. recommender.py : Contains the TechStackRecommender class. Loads the dataset, vectorizes skills using TF-IDF, and computes cosine similarity.
3. dataset/raw_skills.csv : CSV file mapping 21 tech careers to their required skills.
4. requirements.txt : Lists Python dependencies (pandas, scikit-learn, numpy).

3. Input Screenshot

The application launches with a welcome banner and prompts the user to enter their name followed by at least three skills or interests. Below is a captured terminal session showing the input phase.

```
==================================================
Welcome to AI Tech Stack Recommender
==================================================
Enter your name: Alice

Hello Alice!

Please answer a few questions.

Enter at least THREE skills/interests.
Skill 1: Python
Skill 2: Machine Learning
Skill 3: Django
```

Figure 1: Input Phase - User enters name and skills

4. Output Screenshot

After processing, the system displays the top 3 career recommendations ranked by cosine similarity score. Each recommendation includes the career name, a similarity score between 0 and 1, and the required skills.

```
============================================================
Top Career Recommendations for Alice
============================================================

1. AI Engineer
   Similarity Score : 0.436
   Required Skills  : Python Machine Learning Deep Learning
                      TensorFlow PyTorch NLP Computer Vision
                      Git Linux

2. Data Scientist
   Similarity Score : 0.387
   Required Skills  : Python Pandas NumPy Statistics Machine
                      Learning SQL Data Visualization

3. Machine Learning Engineer
   Similarity Score : 0.366
   Required Skills  : Python Scikit-learn TensorFlow PyTorch
                      Machine Learning Feature Engineering

Thank you for using the recommender!
```

Figure 2: Output Phase - Top 3 Career Recommendations

5. Dataset Overview

The dataset (dataset/raw_skills.csv) contains 21 tech career entries. Each entry has a Career name and a space-separated list of associated Skills. Below is the complete dataset used by the system:

Career	Skills
AI Engineer	Python Machine Learning Deep Learning TensorFlow PyTorch NLP
Data Scientist	Python Pandas NumPy Statistics Machine Learning SQL Data Vis
Data Analyst	Excel SQL Power BI Python Tableau Statistics Data Cleaning
Backend Developer	Python Django Flask REST API SQL Git Docker
Frontend Developer	HTML CSS JavaScript React Bootstrap UI UX
Full Stack Developer	HTML CSS JavaScript React Node.js Express MongoDB SQL Git
Cloud Engineer	AWS Azure Google Cloud Docker Kubernetes Linux Networking Te
DevOps Engineer	Docker Kubernetes Jenkins Git Linux AWS CI/CD Terraform
Cyber Security Analyst	Networking Linux Ethical Hacking Penetration Testing SIEM Fi
Automation Engineer	Python Selenium Automation Testing Robot Framework CI/CD
ML Engineer	Python Scikit-learn TensorFlow PyTorch Machine Learning Feat
Mobile App Developer	Flutter Dart Firebase Android iOS REST API
Software Engineer	Java Python C++ OOP Git Data Structures Algorithms SQL
Database Administrator	SQL MySQL PostgreSQL Oracle Database Backup Performance Tuni
Network Engineer	Networking Cisco Routing Switching Linux Firewalls TCP/IP
Game Developer	Unity C# Unreal Engine C++ Game Design Physics
Embedded Systems Enginee	C C++ Microcontrollers Arduino Raspberry Pi IoT Embedded Lin
Blockchain Developer	Solidity Ethereum Smart Contracts Web3 JavaScript Cryptograp
UI UX Designer	Figma Adobe XD Wireframing Prototyping User Research Design
QA Engineer	Manual Testing Automation Selenium JUnit TestNG Bug Tracking

6. Code Explanation

6.1 TechStackRecommender Class (recommender.py)

The core recommendation logic resides in the TechStackRecommender class. It uses scikit-learn's TfidfVectorizer to convert skill text into numerical vectors, then computes cosine similarity between user input and all career entries.

Key method - recommend(user_skills, top_n=3):

```
def recommend(self, user_skills, top_n=3):
    user_profile = ' '.join(user_skills)
    user_vector = self.vectorizer.transform([user_profile])
    similarity = cosine_similarity(
        user_vector, self.skill_vectors
    ).flatten()
    self.data['Score'] = similarity
    recommendations = self.data.sort_values(
        by='Score', ascending=False
    )
    return recommendations.head(top_n)
```

6.2 Entry Point (main.py)

The main.py script orchestrates the user onboarding flow and displays results.

```
def main():
    name, skills = onboarding()
    recommender = TechStackRecommender(
        'dataset/raw_skills.csv'
    )
    results = recommender.recommend(skills, top_n=3)
    for i, row in enumerate(results.itertuples(), 1):
        print(f'{i}. {row.Career}')
        print(f'Similarity Score : {row.Score:.3f}')
        print(f'Required Skills  : {row.Skills}')
```

7. Installation & Usage

7.1 Prerequisites

Python 3.8+ and pip package manager are required.

7.2 Installation Steps

```
pip install -r requirements.txt
```

7.3 Running the Application

```
python main.py
```

7.4 Dependencies

pandas: Data loading and manipulation

scikit-learn: TF-IDF vectorization and cosine similarity

numpy: Numerical operations

8. Future Enhancements

- Web-based UI using Flask or FastAPI for a visual interface
- Expanded dataset with more career paths and skill taxonomies
- Multi-language support for international users
- Skill gap analysis showing missing skills for target careers
- Integration with job APIs (LinkedIn, Indeed) for live job matching
- User profile persistence and recommendation history